

CPSC 250

Class Customization

Edward Brash

1 Introduction

One of the most common things we want to do with an object is print information about it.

For example, suppose we have:

```
moes = Restaurant("Moe's", "Mexican")
```

It would be convenient if we could simply write:

```
print(moes)
```

and automatically obtain useful information about the object.

Python provides a special mechanism for doing exactly this.

2 The `__str__()` Method

Python classes can define a special method:

```
def __str__(self):
```

This method tells Python how the object should appear when printed.

The method should return a string.

Example:

```
def __str__(self):  
    return f"{self.name} ({self.cuisine})"
```

3 Automatic Behavior

Suppose we define:

```
class Restaurant:  
  
    def __init__(self, name, cuisine):  
        self.name = name  
        self.cuisine = cuisine  
  
    def __str__(self):  
        return f"{self.name} ({self.cuisine})"
```

Then:

```
moes = Restaurant("Moe's", "Mexican")  
  
print(moes)
```

automatically calls:

```
moes.__str__()
```

behind the scenes.

4 Important Note

The `__str__` method must return a single string.

It does not directly print the information itself.

Bad design:

```
def __str__(self):  
    print(self.name)
```

Correct design:

```
def __str__(self):  
    return self.name
```

The returned string is then sent to the `print()` statement.

5 Benefits of `__str__()`

The `__str__` method makes objects easier to work with.

Advantages include:

- cleaner debugging,
- easier printing,
- more readable output,
- more professional object design.

6 Example

Suppose we define:

```
class Restaurant:  
  
    def __init__(self, name, cuisine):  
        self.name = name  
        self.cuisine = cuisine  
  
    def __str__(self):  
        return f"Restaurant: {self.name}, Cuisine: {self.cuisine}"
```

Then:

```
moes = Restaurant("Moe's", "Mexican")  
  
print(moes)
```

produces:

```
Restaurant: Moe's, Cuisine: Mexican
```

7 Customizing Objects

This is our first example of **class customization**.

Python allows us to redefine how many built-in operations behave for our own classes.

This leads naturally to operator overloading.

8 Transition to Operator Overloading

A natural question is:

What does it mean to add two objects together?

For example:

```
moes = Restaurant()  
panera = Restaurant()  
  
fusion = moes + panera
```

What should that mean?

The answer depends on the class.

For our own classes, we can define the rules ourselves.

9 Algebra and Operators

Algebra is really a set of rules for manipulating symbols and objects.

Operators include:

- arithmetic operators:

$+, -, *, /, \%$

- comparison operators:

$<, >, <=, >=, ==, !=$

Python allows us to redefine these operators for our own classes.

10 Example: A Time Class

Suppose we define a Time class:

```
class Time:

    def __init__(self, hours, minutes):
        self.hours = hours
        self.minutes = minutes

    def __str__(self):
        return f"{self.hours}:{self.minutes}"
```

Now we can print time objects naturally.

```
t1 = Time(10, 40)

print(t1)
```

11 Comparison Operators

Suppose we want to compare two time objects.

For example:

```
time1 < time2
```

Python does not automatically know how to compare two custom objects.
We must define the comparison ourselves.

12 The __lt__() Method

The special method:

```
def __lt__(self, other):
```

defines the meaning of the less-than operator:

<

for objects of the class.

13 Example Implementation

```
class Time:

    def __init__(self, hours, minutes):
        self.hours = hours
        self.minutes = minutes

    def __str__(self):
        return f"{self.hours}:{self.minutes}"

    def __lt__(self, other):
```

```

    if self.hours < other.hours:
        return True

    elif self.hours == other.hours:

        if self.minutes < other.minutes:
            return True

    return False

```

14 Using the Comparison

Now suppose:

```

time1 = Time(10, 40)
time2 = Time(12, 15)
time3 = Time(9, 15)

```

We can search for the earliest time:

```

min_time = time1

if time2 < min_time:
    min_time = time2

if time3 < min_time:
    min_time = time3

print(f"Earliest time is {min_time}")

```

Because we defined `__lt__`, Python now knows how to compare `Time` objects.

15 Rich Comparison Operators

Python provides several comparison customization methods.

Method	Operator
<code>__lt__</code>	<code><</code>
<code>__le__</code>	<code><=</code>
<code>__gt__</code>	<code>></code>
<code>__ge__</code>	<code>>=</code>
<code>__eq__</code>	<code>==</code>
<code>__ne__</code>	<code>!=</code>

16 Important Warning

Some comparison methods have default behavior, but that behavior may not match what we actually want.

Therefore:

Be careful and define comparison behavior explicitly whenever meaningful comparisons are needed.

17 Key Takeaways

- The `__str__` method controls how objects print.
- `print(object)` automatically calls `object.__str__()`.
- Operator overloading allows us to redefine operators for custom classes.
- Comparison operators are implemented using special methods such as `__lt__`.
- Different classes may define operators in different ways.